

OMNISTOCK

Frontend Product Requirements Document

Multi-Warehouse Inventory Control & Stock Reconciliation System

PS017 · Complete Web Application Specification

A production-oriented frontend specification for a role-based React/TypeScript application integrated through the OmniStock API Gateway.

Version 1.0 · September 2026

1. Product Overview

OmniStock is a multi-warehouse inventory control platform. The frontend must expose the system's core capabilities through a clear, responsive and professional web experience for Customer, Warehouse Manager and Administrator roles.

The frontend is a single React/TypeScript application communicating through the API Gateway. The existing implementation plan defines Gateway, authentication, product, inventory and order services.

■filecite■turn0file0■L30-L60■

Product Vision

Create a polished enterprise SaaS interface combining e-commerce for customers with an operational control center for managers and administrators.

UX Principles

- Clarity of stock, warehouse and order status.
- Role-specific navigation and actions.
- Prominent low-stock, discrepancy and failure states.
- Responsive desktop-first management UX.
- Backend inventory remains the source of truth.
- Explicit loading, success, empty and error states.

2. Goals and Non-Goals

Goals

- Secure JWT login and role-aware access.
- Customer browsing, cart, checkout and order tracking.
- Management dashboards for products, warehouses, inventory, orders and alerts.
- First-class reconciliation and transaction-history workflows.
- Gateway-only backend integration.
- Reusable tables, forms, charts, cards and notifications.

V1 Non-Goals

- Separate native mobile app.
- Payment processing unless later exposed by backend.
- Replacing backend authorization/inventory validation with client logic.
- Separate frontend application per microservice.

3. Users and Role Experiences

Role	Primary goals	Interface
Customer	Discover, buy and track products	E-commerce
Warehouse Manager	Control stock, warehouses, reconciliation and orders	Operations dashboard
Administrator	Manage and monitor the platform	Full management dashboard

Customer navigation

Home · Products · Categories · Cart · My Orders · Profile · Logout

Manager navigation

Dashboard · Products · Warehouses · Inventory · Low Stock · Reconciliation · Transactions · Orders · Profile · Logout

Admin navigation

Dashboard · Products · Warehouses · Inventory · Low Stock · Reconciliation · Transactions · Orders · Users · System Overview · Profile · Logout

4. Information Architecture

Area	Screens
Authentication	Login, Registration, session/error states
Management	Dashboard, Products, Warehouses, Inventory, Low Stock, Reconciliation, Transactions, Orders
Customer	Home, Product Listing, Product Details, Cart, Checkout, Confirmation, My Orders, Order Details
Administration	Users, System Overview, Profile

5. Visual Design System

Design direction

Modern SaaS + enterprise operations dashboard: generous whitespace, rounded cards, thin borders, restrained shadows, strong hierarchy and excellent table usability.

Semantic colors

Semantic	Use
Primary blue/indigo	Navigation, primary actions, active states
Success green	Healthy stock, confirmed/success states
Warning amber	Low stock, pending, attention
Danger red	Critical stock, failures, destructive actions
Slate/neutral	Text, borders, informational UI

Components

- Cards for KPIs.
- Badges for stock/order status.
- Drawers/modals for forms.
- Confirmation dialogs for destructive actions.
- Toast notifications for short feedback.
- Consistent typography and spacing.

6. Global Application Shell

Management

Persistent sidebar + top bar + main content. Top bar contains page context, notifications, search where appropriate and profile.

Customer

Top navigation with branding, product/category navigation, search, cart and account controls.

Responsive

- Sidebar becomes a drawer.
- Wide tables become scrollable/responsive.
- Dashboard cards/charts stack on small screens.
- Primary actions remain reachable.

7. Authentication and Access Control

Login

Validate credentials, call authentication through the Gateway, establish session/token and redirect according to role.

JWT handling

- Centralize token attachment in the HTTP client/interceptor.
- Send Authorization: Bearer on protected requests.
- On 401, clear session and redirect to login.
- Never duplicate authorization logic across components.

Route guards

Protected routes require authentication; restricted routes require the correct role. UI hiding is convenience only—backend authorization remains authoritative.

8. Customer Experience

Home

- Product search/hero area.
- Featured/recent products.
- Category shortcuts.
- Fast access to cart and orders.

Product Listing

Search, category filtering, pagination and sorting where supported. Cards show product name, SKU where appropriate, price and availability.

Product Details

Show product information and multi-warehouse availability when exposed by the API.

Cart

Line items, quantities, prices, totals, availability warnings and remove/update actions.

Checkout

Concise review-and-place-order flow. Backend is authoritative for reservation and stock.

Concurrency failure

If stock changes before checkout, show a specific stock-conflict message and let the customer update the cart.

Orders

Confirmation page plus searchable order history and order details with status/timeline, items, quantities and totals.

9. Manager/Admin Dashboard

KPI cards

- Total Products
- Total Warehouses
- Total Stock
- Reserved Stock
- Available Stock
- Low Stock Items
- Pending Orders
- Confirmed Orders

Analytics

- Inventory by warehouse.
- Stock movement over time.
- Order volume/status.
- Low-stock distribution.

Operational alerts

Critical and low-stock alerts must be prominent and actionable.

10. Product Management

Searchable, paginated product table with create, view, edit and delete actions according to role.

Element	Requirement
Search	Name/SKU search with debouncing where useful
Filters	Category and backend-supported filters
Table	Name, SKU, category, price and supported metadata
Create/Edit	Validated form
Delete	Confirmation and error handling
Details	Product data plus relevant inventory visibility

11. Warehouse Management

- Warehouse name/identifier and supported location data.
- Stock/product summary when available.
- Capacity visualization only if backend provides capacity data.
- View Inventory action.
- Create/edit/delete according to permissions.

12. Inventory Management

Inventory list

Filter by warehouse, product, category and stock status. Show total, reserved and available quantities.

Status	Meaning
Healthy	Above operational threshold
Low Stock	Below configured threshold
Critical	Very low stock
Out of Stock	No available quantity

Inventory details

- Product and warehouse context.
- Total/reserved/available quantity.
- Threshold where supported.
- Stock adjustment.
- Reconciliation.
- Transaction history.

13. Low Stock Management

Dedicated operational screen with Critical/Low/Normal filters where applicable. Show product, warehouse, available quantity, threshold, severity and quick navigation/replenishment action where supported.

14. Inventory Reconciliation

The reconciliation screen must make system quantity, physical quantity and difference unmistakable.

Field	Behavior
Warehouse	Required selector
Product	Required selector
System quantity	Read-only backend value
Physical quantity	Required numeric input
Difference	Computed display; backend authoritative
Reason	Based on backend validation
Notes	Optional text

Field	Behavior
Submit	Confirmation before commit

After success, show old quantity → new quantity, adjustment, timestamp and user context when available.

15. Transactions and Audit History

- Filter by warehouse, product, type and date where supported.
- Show timestamp, product, type, quantity, warehouse and reference.
- Differentiate sale, reservation, release, restock, reconciliation and other backend-supported types.
- Provide useful empty state.

16. Order Management

Manager/Admin order list

Filters for status/date/search. Show order ID, reference/customer, item count, amount and status.

Order details

Show metadata, items, quantities, total, status history and relevant reservation information where exposed.

Failure UX

Display clear failure state. Compensating transactions remain backend responsibility.

17. API Integration

Frontend communicates through the API Gateway. The existing plan defines Gateway 8080 and separate Auth/Product/Inventory/Order services.
filecite turn0file0L30-L60

Domain	Capability
Auth	Registration/login/session calls
Product	CRUD, retrieval, search/filtering as supported
Inventory	Warehouses, stock, reservations, release/confirm, reconciliation, low stock, transactions
Order	Order placement, retrieval and status

Exact paths, query parameters and request/response bodies must match the implemented backend/OpenAPI contract; this PRD does not invent unsupported endpoints.

18. Frontend Architecture

Recommended stack: React + TypeScript + Vite, React Router, Axios, TanStack Query, React Hook Form + Zod, Tailwind CSS and Recharts.

Layer	Responsibility
Pages	Route-level screens
Components	Reusable UI elements
API	Central HTTP client and domain modules
Hooks	Data fetching/mutations and reusable behavior

Layer	Responsibility
Auth	Session/user/role and route protection
Types	Shared TypeScript domain/API models

Suggested structure: src/api · components · layouts · pages · hooks · context · routes · types · utils · App.tsx · main.tsx

19. State and UX Behavior

- Use server-state management for fetching, caching, mutations and invalidation.
- Use local state for temporary UI concerns.
- Use skeletons/table indicators/button pending states.
- Provide meaningful empty states.
- Provide human-readable errors and field validation.

20. Validation and Error Handling

- Client validation for fast feedback; backend remains authoritative.
- Handle 401/403/404/409/422/5xx distinctly where meaningful.
- Treat concurrency conflicts as actionable states.
- Confirm destructive actions.
- Prevent duplicate submissions.

21. Accessibility

- Keyboard navigation and visible focus.
- Labels for controls.
- Meaningful button/link text.
- Color is not the only status indicator.
- Adequate contrast.
- Accessible tables, dialogs and notifications.

22. Security

- Protected requests require authentication.
- Frontend role checks never replace backend authorization.
- No secrets/database credentials in frontend.
- Gateway URL configurable via public environment configuration.
- Do not log JWTs or credentials.

23. Performance

- Lazy-load route-level screens where useful.
- Debounce searches.

- Use server-side pagination for large datasets.
- Cache stable queries and invalidate after mutations.
- Avoid rendering huge unpaginated tables.

24. Notifications and Feedback

Event	Feedback
Product mutation	Success toast + refresh/invalidation
Inventory update	Success toast + updated quantity
Reconciliation	Success summary + transaction visibility
Order placed	Confirmation + order reference
401	Session reset/login redirect
403	Permission message
Stock conflict	Specific message + cart action
Server error	Clear error + safe retry

25. Route Specification

Route	Role	Screen
/login	Public	Login
/register	Public	Registration
/	Customer/redirect	Home or role dashboard
/products	Customer/Manager/Admin	Product list
/products/:id	Customer/Manager/Admin	Product details
/cart	Customer	Cart
/checkout	Customer	Checkout
/orders	Customer/Manager/Admin	Orders
/orders/:id	Customer/Manager/Admin	Order details
/dashboard	Manager/Admin	Dashboard
/warehouses	Manager/Admin	Warehouses
/inventory	Manager/Admin	Inventory
/inventory/:id	Manager/Admin	Inventory details
/low-stock	Manager/Admin	Low stock
/reconciliation	Manager/Admin	Reconciliation
/transactions	Manager/Admin	Transactions
/users	Admin	Users
/system	Admin	System overview
/profile	Authenticated	Profile

26. Core User Flows

Customer purchase

Login → Browse/Search → Product Details → Add to Cart → Cart → Checkout → Backend reservation/order → Confirmation → My Orders.

Manager stock control

Dashboard → Warehouse → Inventory → Review → Update/replenish or reconcile → Transactions → Low-stock monitoring.

Reconciliation

Inventory → Warehouse → Product → System quantity → Physical quantity → Difference → Confirm → Success → Transactions.

Order conflict

Checkout → Place order → Conflict/failure → Explain → Refresh stock → Revise cart/retry safely.

27. Testing Requirements

- Component and form validation tests.
- Authentication and route-guard tests.
- API error handling tests.
- Role-based navigation tests.
- Customer checkout tests.
- Inventory/reconciliation tests.
- Concurrency/conflict UX tests.
- Responsive and accessibility checks.
- End-to-end smoke test.

28. Deployment and Environment

Support local, staging and production configurations. The API Gateway base URL must be configurable without changing source code. The backend plan includes Docker and local multi-service verification, so frontend documentation must explain how it connects to the Gateway. ■filecite■turn0file0■L63-L84■

29. Definition of Done

- All required routes exist and are protected.
- JWT integration works against the actual backend.
- Customer can browse, cart, checkout and view orders.
- Manager/Admin can manage products, warehouses and inventory.
- Low stock, reconciliation and transactions use real API data.
- Order management works.
- Loading/empty/validation/auth/conflict/server-error states exist.
- All backend calls go through Gateway.

- Responsive UI is implemented.
- Production build succeeds.
- README documents setup/environment/backend dependency.
- UI clearly demonstrates multi-warehouse inventory control.

30. Recommended Build Order

Phase	Scope
1	Project setup, design system, routing, API client, layouts
2	Authentication, JWT and role guards
3	Products
4	Warehouses and inventory
5	Low stock, reconciliation, transactions
6	Customer cart, checkout, orders
7	Management orders and analytics
8	Errors, responsive UX, accessibility and polish
9	Integration tests, production build and documentation

31. Final UX Standard

The finished frontend should feel like a credible inventory SaaS product—not a basic student CRUD demo.

Products → Warehouses → Inventory → Reservations → Orders → Reconciliation → Transactions

Customer screens should feel like modern e-commerce; manager/admin screens should feel like an operational control center. Both experiences share the same application, authentication and backend while using role-specific navigation and priorities.

The existing implementation plan explicitly covers the six-service architecture and end-to-end verification from registration/login through product, warehouse/stock and order workflows; the frontend should translate those capabilities into a coherent user journey. ■filecite■turn0file0■L63-L84■